ANIME DATA PRIVACY LAB

Python Coding Laboratory Workbook

k-Anonymity | l-Diversity | t-Closeness

Student Name:
Segotier, Claveria, Reyes

Section / Course:
BS INFO 3C

Date Submitted:

Introduction to the Lab Workbook

This workbook contains six (6) hands-on laboratory activities covering three foundational data anonymization techniques: **k-Anonymity, l-Diversity,** and **t-Closeness.** Each technique has two activities of increasing complexity.

All datasets and scenarios are themed around anime fandom, character databases, streaming platforms, and fan communities. Each lab is implemented in Python — you will write, run, and reason through real code.

Learning Objectives

- Implement k-Anonymity, l-Diversity, and t-Closeness algorithms in Python from scratch.
- Use pandas DataFrames to manipulate, group, and verify anonymized datasets.
- Detect equivalence class violations programmatically and apply generalizations.
- Simulate linkage attacks and understand their defenses through code.
- Compute Earth Mover's Distance (EMD) to verify t-Closeness compliance.

How to Use This Workbook

Each lab starts with a scenario and a provided dataset (as a Python dictionary/list). Fill in the marked sections (# TODO) in each code skeleton. Run your code and verify the output matches the expected results. Reflection questions ask you to reason about your implementation and results. Partial credit is awarded for correct logic even if there are minor bugs.

Required Libraries

```
# Install before starting the labs:
pip install pandas numpy scipy

# Import in every lab file:
import pandas as pd
import numpy as np
from collections import Counter
```

Section 1: k-Anonymity

Concept Recap

k-Anonymity ensures that any released record is indistinguishable from at least k-1 other records. Quasi-identifiers (QIs) — attributes that, when combined, can re-identify a person — are generalized or suppressed until every group of identical QI values (called an equivalence class, or EC) contains at least k records.

Lab 1.1 — AnimeVault Fan Registration Database

Scenario

AnimeVault is a popular anime streaming platform. After a data breach scare, their legal team mandated that all user analytics datasets shared with third-party advertisers must satisfy 3-Anonymity (k = 3). You are a junior data engineer. Your task is to implement a Python program that generalizes the quasi-identifiers Age, Region, and Favorite Genre until k=3 is satisfied.

Part A — Dataset

User ID	Age	Region	Favorite Genre	Subscription Plan
U001	17	NCR	Shonen	Basic
U002	18	NCR	Shonen	Premium
U003	17	Cebu	Isekai	Basic
U004	19	NCR	Romance	Premium
U005	22	Davao	Shonen	Standard
U006	22	Davao	Isekai	Basic
U007	25	NCR	Horror	Premium
U008	24	Cebu	Romance	Standard
U009	23	Cebu	Horror	Basic
U010	19	NCR	Isekai	Standard
U011	18	Davao	Shonen	Basic
U012	25	Davao	Romance	Premium

Part B — Build the Dataset in Python

Represent the dataset above as a Python dictionary and load it into a pandas DataFrame.

```
import pandas as pd

# TODO: Fill in the data dictionary below using the table above.
data = {
    'user_id':   [...],    # list of 12 user IDs
    'age':       [...],    # list of 12 integer ages
    'region':    [...],    # list of 12 region strings
    'genre':     [...],    # list of 12 genre strings
    'sub_plan':  [...],    # list of 12 subscription plan strings
}

df = pd.DataFrame(data)
print(df.head())
```

Hint

Use pd.DataFrame(data) to create the DataFrame from a dictionary.
Each key maps to a Python list. Make sure all lists have the same length (12).
After creating df, call df.dtypes to confirm 'age' is read as int64.

Part C — Implement Generalization Functions

Write three Python functions — one for each quasi-identifier — that apply the generalization hierarchies shown below.

Generalization Hierarchies

```
Age:  Exact value → 5-year range → 10-year range → 'Any'
       e.g.  17 → '[15-19]' → '[10-19]' → 'Any'
Region: NCR/Cebu → 'Luzon/Visayas' → 'Philippines'
         Davao → 'Mindanao' → 'Philippines'
Genre:  Shonen/Isekai → 'Action/Adventure' → 'Anime'
         Romance/Horror → 'Drama/Thriller' → 'Anime'
```

```python
def generalize_age(age, level=1):
    """
    Returns a generalized string for the given age.
    level=1 -> 5-year range, level=2 -> 10-year range, level=3 -> 'Any'
    """
    # TODO: Implement this function.
    # Hint: Use integer division (age // 5) to compute 5-year buckets.
    # Example: age=17, level=1 should return '[15-19]'
    pass


def generalize_region(region, level=1):
    """
    Returns a generalized string for the given region.
    level=1 -> island group, level=2 -> 'Philippines'
    """
    # TODO: Implement this function.
    # Hint: Use a mapping dict: {'NCR': 'Luzon', 'Cebu': 'Visayas', 'Davao': 'Mindanao'}
    pass


def generalize_genre(genre, level=1):
    """
    Returns a generalized string for the given genre.
    level=1 -> broad category, level=2 -> 'Anime'
    """
    # TODO: Implement this function.
    Pass
```

Hints & Pointers

For generalize_age at level=1: low = (age // 5) * 5 and high = low + 4 gives the bracket.
For a mapping dictionary: region_map = {'NCR': 'Luzon', 'Cebu': 'Visayas', 'Davao': 'Mindanao'}
For genres, define a similar dict mapping each genre to its broad category.
Use level as a condition: if level == 1 ... elif level == 2 ... else return 'Any'/'Philippines'/'Anime'

Part D — Apply Generalizations and Check k=3

Write a function check_k_anonymity(df, qi_cols, k) that groups the DataFrame by qi_cols and verifies every group has at least k records. Then apply your generalization functions at level=1 and check.

```python
def check_k_anonymity(df, qi_cols, k):
    """
    Returns True if ALL equivalence classes have >= k records.
    Also prints each EC and its size.
    """
    # TODO: Group df by qi_cols and count records per group.
    # Hint: df.groupby(qi_cols).size() returns a Series of counts.
    # Iterate over it and print each group's QI values and count.
    # Return True only if the minimum count >= k.
    pass


# Apply level-1 generalization to a copy of the DataFrame
df_anon = df.copy()
df_anon['age']    = df_anon['age'].apply(lambda x: generalize_age(x, level=1))
df_anon['region'] = df_anon['region'].apply(lambda x: generalize_region(x, level=1))
df_anon['genre']  = df_anon['genre'].apply(lambda x: generalize_genre(x, level=1))

qi = ['age', 'region', 'genre']
```

```
result = check_k_anonymity(df_anon, qi, k=3)
print(f'k=3 satisfied: {result}')

# TODO: If result is False, increase generalization levels and try again.
```

Hints

df.groupby(qi_cols).size() → a Series where index = group tuples, values = counts.
To find the minimum: groups.min() < k tells you if any EC is too small.
Use .reset_index(name='count') to turn the groupby result into a readable DataFrame.
Try level=1 first. If k=3 is not met, increment the level and re-apply all three functions.

Part E — Verify and Display Equivalence Classes

Display the final anonymized dataset alongside each equivalence class, its size, and whether k=3 is met.

```
# TODO: Print (or tabulate) equivalence classes from your final anonymized df.
# Expected output format example:
#   Age      | Region   | Genre            | Count | k=3?
#   [15-19]  | Luzon    | Action/Adventure |   3   | YES
#   ...

ec_summary = df_anon.groupby(qi).size().reset_index(name='count')
ec_summary['k=3_satisfied'] = ec_summary['count'] >= 3
print(ec_summary.to_string(index=False))
```

Part F — Reflection Questions (Written Answers)

F1. What information was lost when you generalized Age and Region? Is this acceptable? Why or why not?

Your answer:

Age changed from exact ages into 10-year ranges, and Region changed from a city/island-area label into "Philippines". This loses geographic and age precision, but it is acceptable for advertiser analytics because broad demographic patterns are still visible while individual identity is better protected.

F2. Suppose an attacker knows User U007 is aged 25, from NCR, and likes Horror. Can they still re-identify the user in your final anonymized dataset? Examine the equivalence class programmatically.

Answer:

U007 cannot be uniquely identified in the final dataset. U007 is hidden inside a group of six users who all share the same anonymized age, region, and genre values. Because there are six matching records, an attacker cannot tell which one is U007 using only those columns.

```
# TODO: Filter df_anon to find which EC would contain U007 (after generalization).
# Print all records in that EC. Can you uniquely identify U007?
u007_age    = generalize_age(25, level=?)      # fill in your chosen level
u007_region = generalize_region('NCR', level=?)
u007_genre  = generalize_genre('Horror', level=?)

ec_u007 = df_anon[
    (df_anon['age'] == u007_age) &
    (df_anon['region'] == u007_region) &
    (df_anon['genre'] == u007_genre)
]
print(ec_u007)
```

F3. What is the main weakness of k-Anonymity exposed when all records in an EC share the same Subscription Plan value? Write 2-3 sentences.

Your answer:

k-anonymity does not protect against homogeneity attacks. If every row in an EC has the same Subscription Plan, an attacker can infer that plan even when they cannot identify the exact row.

Rubric — Lab 1.1

Part B — Correct DataFrame construction: 10 pts
Part C — Correct generalization functions (all 3): 25 pts
Part D — Correct check_k_anonymity implementation: 20 pts
Part E — Correct EC display: 15 pts
Part F — Reflection (10 pts each): 30 pts

TOTAL: 100 pts

Lab 1.2 — Attack Scenario: Re-Identifying Characters from HeroRank Dataset

Scenario

HeroRank is a fan-run anime character database that released a k=2 anonymized dataset. You are a security researcher. Using an external 'Anime Wiki' dataset, implement a linkage attack in Python — match characters to HeroRank records using shared quasi-identifiers and reveal their secret weaknesses. Then re-anonymize the dataset to k=4.

Part A — Build Both Datasets in Python

Record #	Power Level Range	Origin Region	Weapon Class	Secret Weakness
R1	8000–9000	Eastern Kingdom	Sword	Fire
R2	8000–9000	Eastern Kingdom	Sword	Ice
R3	5000–6000	Northern Wastes	Magic Staff	Dark Magic
R4	5000–6000	Northern Wastes	Magic Staff	Holy Light
R5	9500–10500	Capital City	Bare Hands	None
R6	9500–10500	Capital City	Spear	Poison
R7	3000–4000	Southern Isles	Bow	Close Combat
R8	3000–4000	Southern Isles	Bow	Thunder

Character Name	Exact Power Level	Hometown	Primary Weapon	Anime Series
Kyo Ashura	8450	Ryugawa City (East)	Katana (Sword)	Shinken Wars
Nami Frost	8720	Ryugawa City (East)	Ice Blade (Sword)	Shinken Wars
Zephyr Moon	5300	Frostholm (North)	Rune Staff (Magic)	Ether Chronicles
Seraphiel	9800	Solaris (Capital)	None (Bare Hands)	Ether Chronicles
Riku Darkwind	3200	Shimaoka (South)	Longbow (Bow)	Archipelago Heroes

```
# TODO: Represent BOTH datasets as DataFrames.

herorank = pd.DataFrame({
    'record':    ['R1','R2','R3','R4','R5','R6','R7','R8'],
    'power_low': [...],       # lower bound of power range (int)
    'power_high':[...],       # upper bound of power range (int)
    'region':    [...],       # origin region string
    'weapon':    [...],       # weapon class string
    'weakness':  [...],       # sensitive attribute
})

wiki = pd.DataFrame({
    'name':       [...],
    'power':      [...],      # exact power level (int)
    'hometown':   [...],
    'weapon_raw': [...],      # raw weapon string from Wiki
})
```

Hint

Store power ranges as separate power_low and power_high integer columns (e.g., 8000 and 9000). The Wiki weapon descriptions are longer ('Katana (Sword)') — you will normalize them in Part B. Normalize region strings too (e.g., 'Ryugawa City (East)' → 'Eastern Kingdom').

Part B — Implement the Linkage Attack

Write a function that, for each character in the Wiki, finds the matching record(s) in HeroRank by comparing power level range, region, and weapon class.

```
# Helper: Normalize wiki weapon to match HeroRank 'weapon' column
def normalize_weapon(raw_weapon):
    """
    e.g. 'Katana (Sword)' -> 'Sword'
         'Rune Staff (Magic)' -> 'Magic Staff'
         'None (Bare Hands)' -> 'Bare Hands'
    """
    # TODO: Extract the part inside parentheses using str.split or re module.
    # Hint: raw_weapon.split('(')[-1].rstrip(')') gives the part in parens.
    pass


def linkage_attack(herorank_df, wiki_df):
    """
    For each wiki character, find matching HeroRank records.
    A match occurs when:
        power_low <= wiki_power <= power_high
        AND region matches (after normalization)
        AND weapon class matches (after normalization)
    Returns a list of dicts with keys: name, matched_record, weakness_revealed
    """
    results = []
    for _, char in wiki_df.iterrows():
        norm_weapon = normalize_weapon(char['weapon_raw'])
        # TODO: Normalize char['hometown'] to match HeroRank region labels.
        norm_region = ...  # use a mapping dict

        # TODO: Filter herorank_df using boolean conditions.
        matches = herorank_df[
            (herorank_df['power_low'] <= char['power']) &
            (char['power'] <= herorank_df['power_high']) &
            (herorank_df['region'] == norm_region) &
            (herorank_df['weapon'] == norm_weapon)
        ]
        # TODO: For each match, record the secret weakness exposed.
        for _, row in matches.iterrows():
            results.append({
                'character':  char['name'],
                'matched_record': row['record'],
                'weakness': row['weakness']
            })
    return results

attack_results = linkage_attack(herorank, wiki)
print(pd.DataFrame(attack_results))
print(f'Characters re-identified: {len(attack_results)} out of {len(wiki)}')
```

Hints & Pointers

Iterate over wiki rows using df.iterrows() — each row gives you a character's attributes.
Boolean indexing: df[(condition1) & (condition2) & (condition3)] filters rows.
If a character matches exactly 1 record, that character is fully re-identified.
Region mapping: {'Ryugawa City (East)': 'Eastern Kingdom', 'Frostholm (North)': 'Northern Wastes', ...}

Part C — Re-Anonymize to k=4

Modify the HeroRank dataset to satisfy k=4. Implement the re-anonymization as a Python transformation and verify it using your check_k_anonymity function from Lab 1.1.

```
def re_anonymize_k4(herorank_df):
    """
    Returns a new DataFrame where QI generalizations satisfy k=4.
    Strategy hints:
        - Merge smaller regions together (e.g., combine Eastern Kingdom + Capital City).
        - Widen power level ranges so more records share the same bucket.
        - Suppress (replace with '*') rare Weapon Classes.
    """
    df = herorank_df.copy()
    # TODO: Apply your chosen generalization strategy.
    # Remember: the sensitive attribute 'weakness' must NOT be changed.
```

```
    return df


herorank_k4 = re_anonymize_k4(herorank)

# Verify k=4 using your function from Lab 1.1
qi = ['power_range', 'region', 'weapon']  # use your new column names
check_k_anonymity(herorank_k4, qi, k=4)
```

⚠ **Important**
There is no single correct answer — any generalization that achieves k=4 is valid.
Document your strategy in comments: explain WHY you merged or widened each attribute.
After re-anonymizing, re-run the linkage attack on herorank_k4. How many characters can still be re-identified?

Part D — Reflection Questions

D1. Is a higher k always better? In 2-3 sentences, discuss the trade-offs between k=2 and k=4 for THIS dataset.

Your answer:

A higher k is not always better because it heavily reduces data usefulness. For this dataset, switching from k=2 to k=4 forces you to widen power ranges, merge regions, and erase specific weapon names, making the character details too vague for useful analysis.

D2. Name one real-world industry where this type of linkage attack is a serious concern. Describe how it maps to this scenario.

Your answer:

Healthcare is an industry where linkage attacks are a serious concern. A public, anonymized hospital dataset can be combined with voter registries or social media profiles—just like combining the Wiki database with HeroRank to uncover individual identities and expose sensitive diagnoses.

Rubric — Lab 1.2

Part A — Correct DataFrame construction (both datasets): 10 pts
Part B — Correct linkage attack implementation and output: 35 pts
Part C — Correct k=4 re-anonymization and verification: 25 pts
Part D — Reflection (15 pts each): 30 pts
TOTAL: 100 pts

Section 2: l-Diversity

Concept Recap

l-Diversity extends k-Anonymity by requiring each equivalence class to contain at least l DISTINCT sensitive attribute values. This prevents homogeneity attacks, where an attacker infers a sensitive attribute because all k records in a class share the same value.

Lab 2.1 — OtakuHealth Clinic Patient Records

Scenario

OtakuHealth clinic wants to publish a k=3 anonymized patient dataset for academic research. Your task: (1) Build the dataset in Python, (2) implement a function to check l=2-Diversity, (3) identify which equivalence classes fail, and (4) propose and apply a fix.

Part A — Dataset (Already k=3 Anonymized)

Patient #	Age Group	City District	Convention Role	Diagnosis

P01	Teen (13-19)	Shibuya	Attendee	Anxiety
P02	Teen (13-19)	Shibuya	Attendee	Anxiety
P03	Teen (13-19)	Shibuya	Attendee	Anxiety
P04	Adult (20-35)	Harajuku	Cosplayer	Back Pain
P05	Adult (20-35)	Harajuku	Cosplayer	Back Pain
P06	Adult (20-35)	Harajuku	Cosplayer	Fatigue
P07	Senior (36+)	Akihabara	Vendor	Hypertension
P08	Senior (36+)	Akihabara	Vendor	Hypertension
P09	Senior (36+)	Akihabara	Vendor	Hypertension
P10	Adult (20-35)	Shibuya	Volunteer	Fatigue
P11	Adult (20-35)	Shibuya	Volunteer	Fatigue
P12	Adult (20-35)	Shibuya	Volunteer	Fatigue

Part B — Build the Dataset and Implement l-Diversity Check

```
import pandas as pd

# TODO: Build the patient DataFrame from the table above.
data = {
    'patient_id':  [...],
    'age_group':   [...],
    'district':    [...],
    'role':        [...],
    'diagnosis':   [...],    # sensitive attribute
}
df = pd.DataFrame(data)


def check_l_diversity(df, qi_cols, sensitive_col, l):
    """
    Checks if every equivalence class has at least l distinct sensitive values.
    Prints a summary table of each EC and whether it passes.
    Returns True if ALL ECs satisfy l-Diversity.
    """
    # TODO: Group by qi_cols.
    # For each group, count distinct values in sensitive_col.
    # Print: QI values | distinct values | count | l satisfied?
    # Return True only if all groups have >= l distinct sensitive values.
    pass


qi = ['age_group', 'district', 'role']
check_l_diversity(df, qi, sensitive_col='diagnosis', l=2)
```

Hints

Group the DataFrame: groups = df.groupby(qi_cols)
Count distinct sensitive values per group: groups[sensitive_col].nunique()
nunique() returns the number of UNIQUE values — this is your l count.
To print distinct values (not just count): groups[sensitive_col].unique()

Part C — Identify Failing ECs and Describe a Homogeneity Attack

Use your check_l_diversity function to identify which ECs fail l=2. Then answer:

```
# TODO: Run the check and collect failing ECs.
# Then: Suppose an attacker knows P07 is a Senior vendor from Akihabara.
# What diagnosis can they infer with certainty from the ORIGINAL dataset?

# Identify the EC for P07:
ec_p07 = df[
    (df['age_group'] == 'Senior (36+)') &
    (df['district'] == 'Akihabara') &
    (df['role'] == 'Vendor')
]
print('EC for P07:')
print(ec_p07)
print('Distinct diagnoses:', ec_p07['diagnosis'].unique())
```

```
# TODO: Is this a homogeneity attack? Write your explanation as a comment.
```

> **Pointer**
> A homogeneity attack occurs when ALL records in an EC share the same sensitive value.
> If ec_p07['diagnosis'].nunique() == 1, that EC is homogeneous — an attacker knows the diagnosis.

Part D — Modify the Dataset to Satisfy l=2

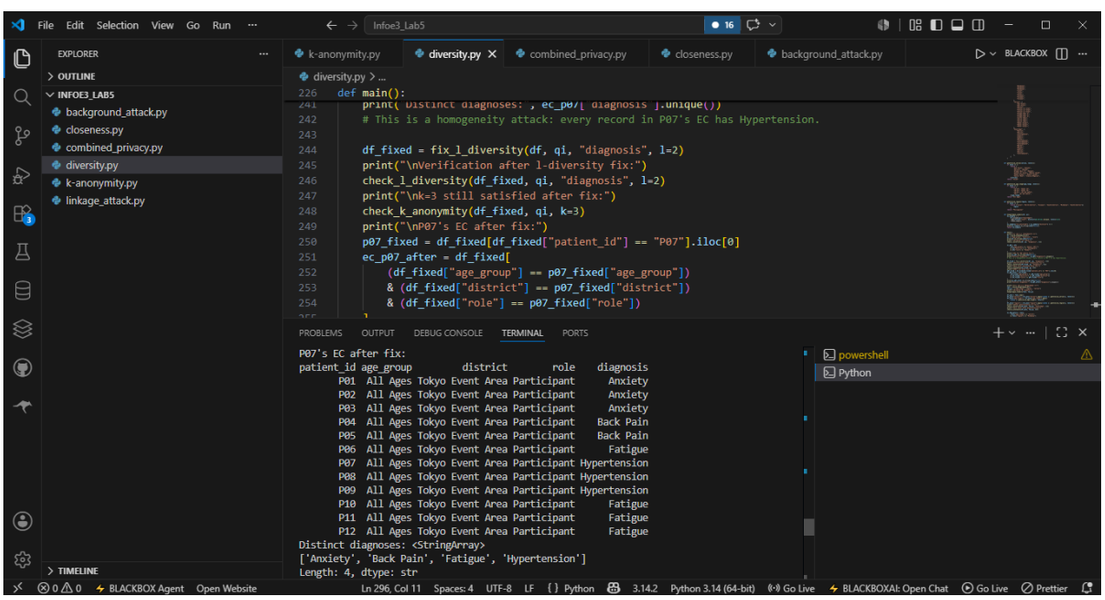
Propose and implement a fix for each failing EC. You may (a) further generalize QIs to merge ECs, or (b) suppress records. Apply your fix and re-verify.

```
def fix_l_diversity(df, qi_cols, sensitive_col, l):
    """
    Returns a modified DataFrame where all ECs satisfy l-Diversity.
    Strategy options:
        (a) Generalize QIs further to merge small homogeneous ECs into larger diverse
ones.
        (b) Suppress (drop) records that make an EC non-diverse.
    Document your strategy in comments.
    """
    df_fixed = df.copy()
    # TODO: Apply your chosen fix.
    # After fixing, call check_l_diversity(df_fixed, qi_cols, sensitive_col, l)
    # to confirm all ECs now pass.
    return df_fixed


df_fixed = fix_l_diversity(df, qi, 'diagnosis', l=2)
print('\nVerification after fix:')
check_l_diversity(df_fixed, qi, 'diagnosis', l=2)

# TODO: Verify P07 is no longer vulnerable.
# Print the EC that P07 now belongs to and list its distinct diagnoses.
```

> **⚠ Design Choice**
> There is no single correct fix. But you must justify your approach in comments.
> Merging ECs (by generalizing a QI) preserves more records but reduces specificity.
> Suppression keeps QI specificity but loses records — bad if the dataset is already small.
> Make sure after fixing, k=3 is STILL satisfied. Call check_k_anonymity on df_fixed too.

Rubric — Lab 2.1

Part B — Correct DataFrame + check_l_diversity implementation: 30 pts
Part C — Correct EC identification and homogeneity attack explanation: 30 pts
Part D — Correct fix + verification + P07 re-check: 40 pts
TOTAL: 100 pts



Lab 2.2 — MangaLeague Fan Tournament Score Sheet

Scenario

MangaLeague publishes anime trivia tournament score sheets for academic study. The sensitive attribute is 'Knowledge Level' (Novice / Intermediate / Expert / Master). Your task: (1) Build equivalence classes programmatically, (2) identify which violate l=3-Diversity, (3) generalize to achieve l=3, and (4) analyze a background knowledge attack.

Part A — Dataset

Fan ID	Age Range	Home Region	Favorite Series	Knowledge Level
F01	15-19	Luzon	One Piece	Novice
F02	15-19	Luzon	One Piece	Novice
F03	15-19	Luzon	Naruto	Intermediate
F04	20-25	Visayas	Attack on Titan	Expert
F05	20-25	Visayas	Attack on Titan	Expert
F06	20-25	Visayas	Attack on Titan	Master
F07	26-35	Mindanao	Dragon Ball Z	Intermediate
F08	26-35	Mindanao	Dragon Ball Z	Intermediate
F09	26-35	Mindanao	Dragon Ball Z	Intermediate
F10	36+	Luzon	Sailor Moon	Master
F11	36+	Luzon	Sailor Moon	Master
F12	36+	Luzon	Sailor Moon	Expert
F13	15-19	Visayas	Demon Slayer	Novice
F14	15-19	Visayas	Demon Slayer	Novice
F15	15-19	Visayas	Demon Slayer	Intermediate

Part B — Build Dataset and Equivalence Class Summary

```
# TODO: Build the DataFrame from the table above.
data = {
    'fan_id':  [...],
    'age_range': [...],
    'region':    [...],
    'series':    [...],
    'knowledge': [...],    # sensitive attribute
}
df = pd.DataFrame(data)

# TODO: Build an EC summary table.
# For each group of (age_range, region, series), compute:
#   - Number of records in the EC
#   - Distinct knowledge levels
#   - Whether l=3 is satisfied (nunique >= 3)

qi = ['age_range', 'region', 'series']
ec_summary = (
    df.groupby(qi)['knowledge']
    .agg(records='count', distinct=pd.Series.nunique, levels=list)
    .reset_index()
)
ec_summary['l3_satisfied'] = ec_summary['distinct'] >= 3
print(ec_summary.to_string(index=False))
```

Hints

pd.Series.nunique counts unique values — use it with .agg().
Pass a list in .agg() to compute multiple aggregations: .agg(count=('knowledge','count'), nunique=('knowledge','nunique'))
To collect all values as a list: .agg(levels=('knowledge', list)) — useful for seeing the distribution.

Part C — Generalize to Achieve l=3

For each EC that fails l=3, apply further generalizations to merge it with other ECs until l=3 is met across all equivalence classes.

```
def generalize_series(series, level=1):
    """
    level=1: group similar series under a genre label
    level=2: collapse all to 'Anime'
    Example groupings (you may define your own):
        One Piece / Naruto / Dragon Ball Z -> 'Shonen'
        Sailor Moon / Demon Slayer / Attack on Titan -> 'Modern Anime'
    """
    # TODO: Implement.
    pass


def generalize_age_range(age_range, level=1):
    """
    level=1: '15-19' and '20-25' -> 'Under 25'
             '26-35' and '36+' -> '26 and above'
    level=2: all -> 'All Ages'
    """
    # TODO: Implement.
    pass


# Apply generalizations and re-check l=3
df_anon = df.copy()
df_anon['series']    = df_anon['series'].apply(lambda x: generalize_series(x, level=1))
df_anon['age_range'] = df_anon['age_range'].apply(lambda x: generalize_age_range(x,
level=1))

check_l_diversity(df_anon, qi, 'knowledge', l=3)
# If still failing, increase level and repeat.
```

> **Strategy Pointer**
> The Dragon Ball Z EC (F07–F09) is the most problematic: all three records have 'Intermediate'.
> You must merge this EC with at least two others that have different knowledge levels.
> Generalizing 'series' from 'Dragon Ball Z' to 'Shonen' would group it with F01–F03 (Luzon, 15-19) — check if that gives >= 3 distinct values.
> After generalizing, re-check BOTH k and l using your previously written functions.

Part D — Background Knowledge Attack

Before and after your generalization, analyze whether F08 is protected.

```
# BEFORE generalization:
ec_f08_before = df[
    (df['age_range'] == '26-35') &
    (df['region'] == 'Mindanao') &
    (df['series'] == 'Dragon Ball Z')
]
print('Before generalization, EC of F08:')
print(ec_f08_before['knowledge'].value_counts())
# TODO: What knowledge level can an attacker infer with 100% certainty?

# AFTER generalization:
# TODO: Filter df_anon to find F08's new EC.
# Print distinct knowledge levels in the new EC.
# Is F08 still uniquely identifiable?
```

Written Question: In your own words, explain the difference between a homogeneity attack and a background knowledge attack. Which one applies to F08 before generalization?

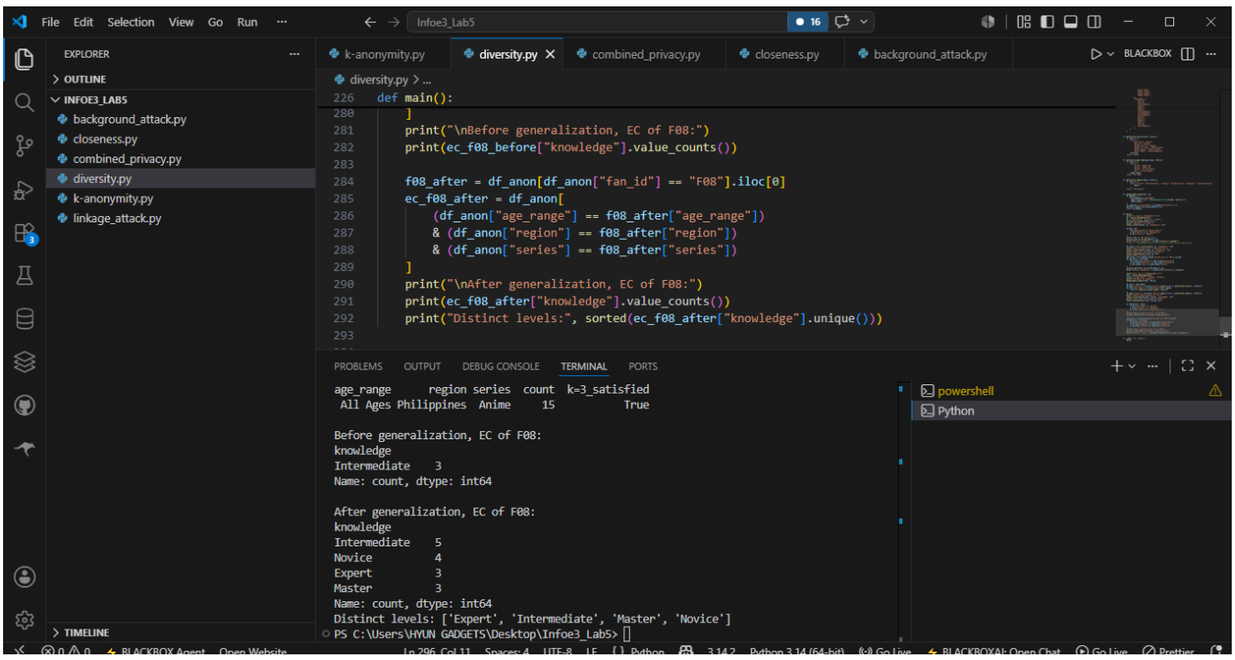
Your answer:

A homogeneity attack happens when all records in an EC share the same sensitive value. A background knowledge attack uses outside facts to narrow down likely sensitive values even if the EC is not fully homogeneous.

F08 before generalization is a homogeneity attack because the Dragon Ball Z/Mindanao/26-35 EC is 100% Intermediate.

Rubric — Lab 2.2

Part B — EC summary table (correct groupby and nunique): 25 pts
Part C — Correct generalization achieving l=3 everywhere: 35 pts
Part D — Before/after code + written explanation: 40 pts
TOTAL: 100 pts



Section 3: t-Closeness

Concept Recap

t-Closeness requires that the distribution of the sensitive attribute within any equivalence class closely matches the overall distribution in the entire dataset. The distance between distributions is measured using the Earth Mover's Distance (EMD). If EMD ≤ t, the EC satisfies t-Closeness.

`EMD for unordered categorical data: EMD = (1/2) × Σ |p_i - q_i|`

Lab 3.1 — NihonStream Viewer Analytics

Scenario

NihonStream is an anime streaming service sharing viewer analytics with research institutions. The sensitive attribute is 'Viewer Tier' (Free / Silver / Gold / Platinum). The dataset has been k=4 anonymized. Your task: implement the EMD calculation in Python and check whether each EC satisfies t=0.25-Closeness.

Part A — Build the Dataset and Compute Overall Distribution

The 20-record dataset has three equivalence classes. Build the full DataFrame and compute the overall distribution of Viewer Tier.

```
import pandas as pd
import numpy as np

# EC1 (n=5): Free, Free, Silver, Free, Silver
# EC2 (n=7): Platinum, Platinum, Platinum, Gold, Gold, Gold, Silver
# EC3 (n=8): Free, Free, Free, Free, Silver, Silver, Gold, Platinum

# TODO: Build the full DataFrame with columns: ['ec', 'viewer_tier']
# Hint: Repeat EC labels for each record (e.g., 'EC1' repeated 5 times, etc.)
```

```
records = []
ec1_tiers = ['Free','Free','Silver','Free','Silver']
ec2_tiers = [...]   # TODO: fill in
ec3_tiers = [...]   # TODO: fill in

for tier in ec1_tiers: records.append({'ec':'EC1', 'viewer_tier': tier})
# TODO: Repeat for EC2 and EC3.

df = pd.DataFrame(records)

# Compute overall distribution
overall_dist = df['viewer_tier'].value_counts(normalize=True).sort_index()
print('Overall distribution:')
print(overall_dist)
```

Hints

value_counts(normalize=True) gives proportions (floats summing to 1.0) instead of raw counts.
.sort_index() ensures consistent ordering (alphabetical) for later comparisons.
Store the overall distribution as a Python dict for easy lookup: overall_dict = overall_dist.to_dict()

Part B — Implement EMD Calculation

Implement the Earth Mover's Distance function for unordered categorical data.

```
def compute_emd_unordered(ec_dist, overall_dist):
    """
    Computes EMD for unordered categorical sensitive attributes.
    Formula: EMD = (1/2) * sum(|p_i - q_i|) for each category i

    Parameters:
        ec_dist (dict): proportion of each category in this EC
        overall_dist (dict): proportion of each category in the full dataset

    Returns:
        float: the EMD value
    """
    # TODO: Implement this function.
    # Step 1: Get all unique categories (union of keys from both dicts).
    # Step 2: For each category, get p_i (EC proportion, default 0 if missing)
    #         and q_i (overall proportion, default 0 if missing).
    # Step 3: Sum |p_i - q_i| over all categories, then multiply by 0.5.
    pass


# Example test (you can verify manually):
test_ec   = {'Free': 0.6, 'Silver': 0.4, 'Gold': 0.0, 'Platinum': 0.0}
test_overall = {'Free': 0.40, 'Silver': 0.25, 'Gold': 0.20, 'Platinum': 0.15}
print(compute_emd_unordered(test_ec, test_overall))  # Expected: 0.175
```

Step-by-Step Walkthrough

Categories = {Free, Silver, Gold, Platinum}
|0.60 - 0.40| = 0.20 (Free)
|0.40 - 0.25| = 0.15 (Silver)
|0.00 - 0.20| = 0.20 (Gold)
|0.00 - 0.15| = 0.15 (Platinum)
Sum = 0.70 → EMD = 0.70 / 2 = 0.35
Use dict.get(key, 0) to safely access keys that may not exist in an EC.

Part C — Check t=0.25-Closeness for Each EC

```
def check_t_closeness(df, ec_col, sensitive_col, overall_dist, t):
    """
    For each unique value of ec_col, compute the EMD and check if <= t.
    Prints a summary table.
    Returns True if ALL ECs satisfy t-Closeness.
    """
    all_pass = True
    for ec_name, group in df.groupby(ec_col):
```

```
        # TODO: Compute ec_dist from the group.
        # Hint: group[sensitive_col].value_counts(normalize=True).to_dict()
        ec_dist = ...
        emd = compute_emd_unordered(ec_dist, overall_dist)
        passes = emd <= t
        if not passes: all_pass = False
        print(f'{ec_name}: EMD={emd:.4f}  t={t}  Pass={passes}')
    return all_pass

overall_dict = df['viewer_tier'].value_counts(normalize=True).to_dict()
check_t_closeness(df, 'ec', 'viewer_tier', overall_dict, t=0.25)
```

Debugging Tip

If your EMD values seem too high or low, print ec_dist and overall_dict side by side for each EC. Make sure both dicts cover ALL categories, defaulting missing ones to 0.
EC2 should fail t=0.25 because it is heavily skewed toward Platinum and Gold, which are rare overall.

Part D — Propose a Fix and Analyze Trade-offs

For the failing EC(s), describe a generalization strategy and implement it. Then answer the trade-off question in code comments.

```
# TODO: Propose and implement a fix for the violating EC.
# Strategy options:
#   (a) Merge EC2 with another EC by further generalizing a QI.
#   (b) Suppress some records from EC2 to rebalance the distribution.
#   (c) Split EC2 into smaller groups (only if k is still satisfied).

# After modifying df, re-run check_t_closeness to verify.

# TODO: Answer in comments:
# Q1: What happens to data utility if t is set very small (e.g., t=0.01)?
# Q2: What privacy risk arises if t is set very large (e.g., t=0.90)?
```

Rubric — Lab 3.1

Part A — Correct DataFrame and overall distribution: 15 pts
Part B — Correct compute_emd_unordered implementation: 30 pts
Part C — Correct check_t_closeness and correct identification of violating EC: 30 pts
Part D — Fix proposal + trade-off analysis comments: 25 pts
TOTAL: 100 pts

Lab 3.2 — AniMed Hospital Fan Event Medical Records

Scenario

AniMed Hospital wants to publish anonymized health records satisfying ALL THREE simultaneously: k=4, l=2, and t=0.30-Closeness. Sensitive attribute: Blood Condition (Normal / Anemia / Pre-diabetic / Hypertensive). This is a DESIGN + VERIFICATION lab: you will design a generalization scheme and verify it meets all three criteria using your Python functions.

Part A — Dataset

Patient ID	Age Group	Cosplay Type	Days Attended	Blood Condition
M01	18-22	Shonen Hero	1 Day	Normal
M02	18-22	Shonen Hero	1 Day	Normal
M03	18-22	Shonen Hero	1 Day	Anemia
M04	18-22	Shonen Hero	1 Day	Anemia
M05	23-30	Magical Girl	2 Days	Normal
M06	23-30	Magical Girl	2 Days	Pre-diabetic
M07	23-30	Magical Girl	2 Days	Hypertensive

M08	23-30	Magical Girl	2 Days	Normal
M09	31-40	Mecha Pilot	3 Days	Hypertensive
M10	31-40	Mecha Pilot	3 Days	Hypertensive
M11	31-40	Mecha Pilot	3 Days	Hypertensive
M12	31-40	Mecha Pilot	3 Days	Pre-diabetic
M13	41+	Classic Villain	2 Days	Normal
M14	41+	Classic Villain	2 Days	Anemia
M15	41+	Classic Villain	2 Days	Hypertensive
M16	41+	Classic Villain	2 Days	Pre-diabetic

Part B — Build the Dataset and Compute Overall Distribution

```
# TODO: Build the DataFrame from the table above.
data = {
    'patient_id':  [...],
    'age_group':   [...],
    'cosplay':     [...],
    'days':        [...],
    'condition':   [...],   # sensitive attribute
}
df = pd.DataFrame(data)

# Compute overall distribution
overall_dist = df['condition'].value_counts(normalize=True).to_dict()
print('Overall distribution of Blood Condition:')
for k_cond, v in sorted(overall_dist.items()):
    print(f'  {k_cond}: {v:.4f}')
```

Part C — Design Generalizations and Assign ECs

You must design a generalization scheme that creates equivalence classes satisfying all three criteria. Implement it as a function that adds a new 'ec' column to the DataFrame.

```
def assign_equivalence_classes(df):
    """
    Adds an 'ec' column to df by generalizing quasi-identifiers.
    Each EC must have: k>=4, l>=2, EMD<=0.30

    Generalization ideas:
        Age: '18-22' + '23-30' -> 'Young Adult (18-30)'
             '31-40' + '41+'   -> 'Middle-Older (31+)'
        Cosplay: 'Shonen Hero' + 'Magical Girl' -> 'Popular Anime'
                 'Mecha Pilot' + 'Classic Villain' -> 'Classic Anime'
        Days: '1 Day'+'2 Days' -> '1-2 Days'
              '3 Days' alone   -> '3 Days'
    You may use ANY valid scheme — justify your choices in comments.
    """
    df = df.copy()
    # TODO: Create generalized columns.
    # TODO: Assign EC labels based on generalized QI combinations.
    # Example: df['ec'] = df['age_gen'] + '_' + df['cosplay_gen']
    return df


df_anon = assign_equivalence_classes(df)
print(df_anon[['patient_id','ec','condition']].to_string(index=False))
```

Design Guidance

Start by checking which ECs violate which criterion — failing one informs your generalization choice.
EC M09–M12 (Mecha Pilot, 31-40, 3 Days) all have 'Hypertensive' or 'Pre-diabetic' — l=2 might be borderline.
To meet t=0.30: Normal ~31%, Anemia ~19%, Pre-diabetic ~25%, Hypertensive ~25% overall — your EC distribution must not deviate too far.
Run all three checks (k, l, EMD) after each attempt and iterate.

Part D — Verify All Three Criteria

Use your functions from Labs 1.1, 2.1, and 3.1 to verify each EC satisfies k=4, l=2, and t≤0.30.

```
qi = ['ec']   # EC label encodes the generalized QI combination

print('=== k=4 CHECK ===')
check_k_anonymity(df_anon, qi, k=4)

print('\n=== l=2 CHECK ===')
check_l_diversity(df_anon, qi, 'condition', l=2)

print('\n=== t=0.30 CHECK ===')
overall_dict = df['condition'].value_counts(normalize=True).to_dict()
check_t_closeness(df_anon, 'ec', 'condition', overall_dict, t=0.30)

# TODO: If any check fails, go back to Part C and revise your generalizations.
# Document each iteration in comments: what failed, what you changed, and why.
```

> ⚠ **Important**
> All three checks must pass simultaneously on the SAME df_anon.
> It is acceptable to iterate 2-3 times before finding a valid scheme — document your attempts.
> If a check function returns False, read its printed output to see WHICH EC is failing.

Part E — Comparative Analysis (Written Answers)

E1. Compare k-Anonymity, l-Diversity, and t-Closeness. For this medical dataset, which provides the strongest privacy protection? Use specific EC examples from your implementation to justify.

Your answer:

k-anonymity gives the base group-size protection, l-diversity ensures each EC has multiple blood conditions, and because it also controls how skewed those conditions are. For example, an EC with four records could pass k and l but still reveal high likelihood of Hypertensive; the EMD check blocks that.

E2. As the privacy officer at AniMed, would you choose t=0.05 or t=0.40? Defend your choice considering both research utility and patient privacy.

Your answer:

I would choose t=0.40 instead of t=0.05 for this small dataset. t=0.05 would force nearly complete merging and harm research utility, while t=0.40 still gives a practical distribution-similarity requirement.

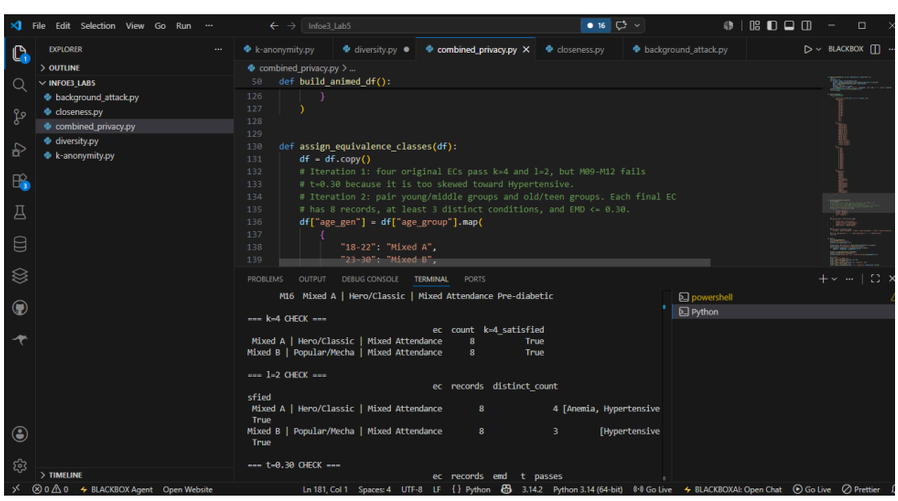
E3. Describe one scenario where t-Closeness alone would still fail to protect a patient's privacy. What additional measure would you combine it with?

Your answer:

t-closeness alone can fail if an attacker already knows a unique QI combination and can isolate a person before checking condition risk. I would combine it with k-anonymity and, for stronger releases, differential privacy on aggregate statistics.

Rubric — Lab 3.2

Part B — Correct DataFrame and overall distribution: 10 pts
Part C — Valid generalization scheme with comments: 30 pts
Part D — All three checks pass + iteration documentation: 30 pts
Part E — Written analysis (10 pts each): 30 pts
TOTAL: 100 pts



Technique Comparison Summary

After completing all six labs, fill in the table and write your synthesis paragraph.

Property	k-Anonymity	l-Diversity	t-Closeness
Core Requirement	Each EC has >= k identical QI records	Each EC has >= l distinct sensitive values	EC distribution matches overall by <= t (EMD)
Prevents	Re-identification / Linkage Attack	Homogeneity Attack, Background Knowledge Attack	Skewness Attack, Similarity Attack
Main Weakness	All records in EC may share one sensitive value	Values may still be skewed in one direction	Difficult to define distance for non-ordered data
Python Function (you wrote)	check_k_anonymity()	check_l_diversity()	compute_emd_unordered() + check_t_closeness()
Lab Used In	1.1 and 1.2	2.1 and 2.2	3.1 and 3.2
Best Used For	k-anonymity is best used for reducing direct linkage from QIs.	l-diversity is best used when the sensitive attribute may be homogeneous.	t-closeness is best used for high-risk attributes with skewed distributions.

Synthesis Paragraph

In 3–5 sentences, explain how these three techniques complement each other when applied together (as in Lab 3.2):

These methods complement each other because each covers a different weakness. k-anonymity makes each person blend into a group, l-diversity prevents the group from having only one sensitive value, and t-closeness checks whether the group's sensitive-value distribution is too revealing compared with the full dataset. Used together, they provide a stronger privacy design than any single technique alone.

References

- Li, N., Li, T., & Venkatasubramanian, S. (2007). t-Closeness: Privacy beyond k-anonymity and l-diversity.
- Machanavajjhala, A., Kifer, D., Gehrke, J., & Venkatasubramanian, M. (2006). l-diversity: Privacy beyond k-anonymity.
- Domingo-Ferrer, J., Soria-Comas, J., & Gonzalez-Nicolas, U. (2016). Enhancing data utility in differential privacy.
- Rajendran, K., et al. (2018). Applications of data anonymization in healthcare and related domains.

All scenarios, datasets, and character names in this workbook are entirely fictional and created for educational purposes.